

1. ①目的・全体概要

- 1.1. 本アプリはST2110およびST2059-2の技術検証のためのSenderおよびReceiverの「Node」を、簡易に再現する。これにより高価な放送機器を購入しなくても、検証および学習を社内ローカルで行うことが可能になる。
- 1.2. サーバーとしては、Ubuntu Server LTS24.04が動く、5年前の中古ラップトップを使用することを想定する。安価かつ簡易に動かすことを忘れない。
- 1.3. フロントエンドはWebアプリとして、人間の操作系はWebGUIとする。そのため本システムではPythonを用いたWebサーバーを構築し、バックエンドではコンテナなどで疑似的なMoIPのNode機能を持たせる。このコンテナの設定変更・ST2110およびST2059-2の状態確認・取得ST2110ストリームやST2059-2のPTP Followerデータ可視化は、このフロントエンドのPythonのWebサーバー経由で人間系と橋渡しする。
- 1.4. 外部のZabbixなどの外部監視ツールからREST APIで本システムの状態を取得して、全社の総合監視ができるようにする
- 1.5. ミドルウェアはclaudeのおすすめを使用する。検証目的を忘れずに信頼性の超高いものではなく、簡易で安価に使えるものを使用する
- 1.6. 本システムでは、原則は放送実用の1920×1080の2K HDのような約1.3Gbpsの高ビットレートな映像データは流さず、第1期では映像に限らず音声もメディアパケットは流さない。第2期になっても、基本は1Gbps NICを利用した、640×480のSD画質（約270Mbps）でのマルチキャスト疎通の正常性をL1~L3レベルで確認することでネットワーク評価を行う。
 - 1.6.1. ただし、高スペックPCを用意した映像評価をL7で人間が主観評価することや、10GbpsNICのハードウェアを用意した放送品質の本格的なL7レベルの評価を行うこと、は機能としては盛り込む。本装置は検証用とであり、マシンスペックでこれが正常に動かないことはユーザーの責任とする
- 1.7. 補足資料「検証用MoIP-Node開発_要件定義書_補足資料」は、本書で文字では説明が難しい内容を図示するときに参照する。都度参照先を本書および補足資料で定義する
 - 1.7.1. https://docs.google.com/presentation/d/1r2_lkf8N7XezmGrjDVjPUO0497gdZornIwjYrYYMvhAo/edit?usp=sharing

2. ②システム要件

2.1. ハードウェア要件・OS・インフラ要件

- 2.1.1. Ubuntu Server LTS 24.04が動くこと
- 2.1.2. 1Gbps以上のNICを3つ以上備えること
- 2.1.3. 本書のアプリケーションが正常に動くかどうかは運用者が自身で動作確認するものとする

2.2. ネットワーク要件

- 2.2.1. 3つのNICの使用方法は下記とする
 - 共通
 - NICは固定IPアドレス or DHCPで運用する
 - IPアドレスの変更はWebGUIのシステム設定から行う
 - 変更後はサーバーOS再起動
 - NIC①: MoIPメディアストリーム ST2022-7のAmber面
 - 後述するST2110のマルチキャストストリームが流れる
 - 後述するST2059-2のPTPマルチキャストストリームが流れる
 - NIC②: MoIPメディアストリーム ST2022-7のBlue面
 - 後述するST2110のマルチキャストストリームが流れる
 - 後述するST2059-2のPTPマルチキャストストリームが流れる
 - NIC③: 制御
 - WebGUI、REST APIなどの外部連携、NMOS制御に関する通信が流れる
- 2.2.2. リンクスピードは基本は1Gbpsとするが、特にNIC①については、10Gbpsを考慮する。
- 2.2.3. ST2022-7の冗長は考慮する(3章にも共通)

3. ③機能要件

3.1. MoIP Node機能(第2期開発)

- 3.1.1. 共通
 - NMOS IS-04 (Ver1.0, 1.1, 1.2, 1.3)に準拠したふるまいをする
 - 上位のNMOS RDSにNodeとしてRegistration APIで登録できる
 - IS-05 (Ver1.0, 1.1)に準拠したふるまいをする
 - 上位のNMOSのコントローラーからConnctcion Managgement APIで制御ができる
- 3.1.2. Sender
 - マルチキャストストリームを出力する。ST2110に則る
 - PTP同期として、本章で定めるST2059-2に準拠した時刻同期を元にしたRTPヘッダーへのタイムスタンプを付与する
 - 機能はOFF/ONできる
 - ST2022-7に則り、同じマルチキャストパケットを、別のマルチキャストグループアドレス宛に2つのNICから同時に送信する
 - ST2022-7の機能は、OFF/ONできる
 - Amberのみ
 - Amber+Blue冗長モード
- 3.1.3. Receiver
 - マルチキャストストリームがIGMP-v3に準拠したマルチキャストプロトコルに則り入力する。ST2110に則る

- 入力されたマルチキャストストリームを評価する。詳細は別途検討する。WebGUI画面にWebRTCなどのストリーミングとする
- PTP同期として、本章で定めるST2059-2に準拠した時刻同期を元にしたRTPヘッダーのタイムスタンプは無視する。ストリームの受信が目的であり、厳密なPTPによる放送機器同期の再現はしない。
- ST2022-7については、冗長受信はしない
 - NIC①とNIC②は別々に受信する
 - ベースバンドの表示は、NIC①(Amber)とNIC②(Blue)のどちらかをWebGUI①で選択する
 - マルチキャストストリームの評価は、NIC①とNIC②で個別に行う
 - あくまでマルチキャストネットワークの正常性確認が大切であり、映像音声の疎通確認が目的ではない
- 機能はOFF/ONできる
 - Receiver機能のOFF/ON切り替えを行う。この際NIC①は必ずOFF/ONが同時に切り替わる。ただし、NIC②は、別にGUIでの切り替えを設けて、個別にOFF/ON切り替えができるようにする
 - Receiver機能をONにすると、NIC①は強制的にONになる。この際のNIC②側は、別のボタンの状態次第でONの場合とOFFの場合がある。
 - Receiver機能がOFFの時は、NIC②のReceiver機能も強制的にOFFになる
- NMOS制御との整合
 - NMOSでのReceiverの制御は、基本的に、Amber/Blueの両方の制御である。本装置においては、前述の機能のOFF/ONで対応する。前述の機能により、NIC①とNIC②がEnableの場合は、NMOSからの制御はそのまま両方のReceiver設定を変更する。もしNIC①のみがONの場合は、NIC ②は設定値は変更するがOFFであるのでIGMP-Joinは出力しない。NIC②のみがONの場合は設計上はあり得ないが、もし発生した場合は、NIC①のみONの時と同様のふるまいとする。

3.2. PTP Follower機能(第1期開発)

- 3.2.1. ST2059-2、IEEE1588に準拠したPTP機能を持つ。放送用であることを忘れない。
- 3.2.2. 物理サーバー1台で1つのFollowerとしてふるまう。いかなる場合でもPTPのLeaderにはならない(安全設計)
- 3.2.3. PTP Followerとして取得した時計はOS時刻とする。本システムで動く全てのプロセス(ST2110のRTPを含む)は、OS時刻を時刻源とする。OSのNTPサーバーからの時刻同期は行わないためOS設定レベルでOFFにする。
 - OS時刻→ログの時刻などに使用する
 - PTP時刻→第2期開発のST2110ストリームでも使用する時刻
- 3.2.4. コミュニケーションモード
 - 「マルチキャスト」のみ
- 3.2.5. Delay Mechanism
 - End-to-End(E2E)のみ
- 3.2.6. NICについて
 - NIC①のAmberのみ有効とする
 - WebGUIについては、GUI①とGUI②、各所に表示させてオペレータに認知させる
- 3.2.7. WebGUIでの設定変更可能項目
 - ドメイン番号
 - Priority設定値

- Priority1
- Priority2
- コミュニケーションの周波数
 - Sync
 - Announce Interval
 - Delay Request Interval

3.2.8. PTPの状態評価のための数値がWebGUI上に表示される

- 表示のリフレッシュ間隔は1秒とする
- PTPロック状態
 - 定義: Offset from Masterが500ns以内で3秒安定 ※コンフィグファイルで変更可能。WebGUIでは変更不可
- GM ID (BMCAにより決定された神様のPTP-SGのID)
- Source ID (本装置が直接繋がっているBCのネットワークスイッチのID)
- Offset from Master
 - 1秒間の平均、最大値、最小値を表示

3.2.9. 計測ログ

- ログ記録の間隔は1分とする
- ログの記録の内容
 - PTPロック状態、GM ID、Source ID
 - 1分に1回の取得した時のものを1発、その時点の状態を記録
 - Offset from Master
 - 1分間の平均、最大値、最小値を計算して1分に1セットのデータとして記録する
 - ログの時刻はサーバーのOS時計とする
 - ログは1日に1ファイルとして分割する

3.2.10. イベントログ

- エラー発生時の内容と時刻を記録する
- エラー定義
 - PTPロック外れ
 - Offset from Masterの閾値越え
 - 閾値はWebGUIで変更可能(デフォルトは500nsとする)
 - PTPパケット周波数乱れ
 - 次のパケットが設定値から外れた3秒以上連続で外れた場合
 - syncパケット
 - announce messageパケット
 - GM ID変更
 - IDが変わったことを記録
 - サンプリング周期はannounce messageと同じ周波数

3.2.11. 外部API

- 外部のZabbixなどの別システムから、PTPの状態をREST APIで取得できる
- セキュリティ
 - 検証用なので考慮不要
- 外部からGETメソッドで来た時のデータを出力
 - PTPロック状態、GM ID、Source ID

- リクエストが来た時の値をそのまま出力
 - Offset from Master
 - リクエストが来た時の、WebGUI用の1秒更新の平均値・最大値・最小値、を出力
 - データは上記のデータをまとめて1つのjsonで出力する

3.3. システムエラー

- 3.3.1. 開発中のデバッグのため、本システム全体でのログを記録・表示する

3.4. WebGUI機能

3.4.1. 共通

- 設定変更はGUI毎に「保存」ボタンでまとめて反映する
- ログイン画面なし・認証なしでOK。

3.4.2. WebGUI一覧

- ダッシュボード WebGUI①
 - 7つのペインを持つ
 - ①-1 機能領域
 - 機能On/Offなど機能設定
 - ①-2 システム領域
 - 設定変更GUIへの遷移ボタン
 - ①-3 PTP領域
 - ①-4 Sender①領域
 - ①-5 Sender②領域
 - ①-4と同じ機能
 - ①-6 Receiver①領域
 - ①-7 Receiver②領域
 - ①-6と同じ機能

- 設定変更 WebGUI②
 - タブで作業領域を分ける

3.4.3. 機能一覧 ダッシュボード・WebGUI①

- 共通
 - 状態は1秒1回の自動リフレッシュとする
- ①-1 機能領域
 - ポップアップを表示
 - 次の機能ON/OFFを設定
 - Node機能
 - NMOS RDSへの通知の部分
 - Sender①の出力
 - Sender②の出力
 - Receiver①の動作
 - Receiver②の動作
- ①-2 システム領域
 - 設定変更 WebGUI②への遷移
 - CPU使用率、メモリー使用率表示
 - システムエラーログ表示のボタン
 - 押すとポップアップで最新100件表示
 - csvエクスポート可能
 - NIC①とNIC②は、帯域利用状況とリンクアップ状況(●Mbpsなどの表示)の表示。
 - NIC③は、制御なのでリンクアップ状況のみ表示。
 - など

- ①-3 PTP領域
 - 本章の「PTP Follower機能」で定めた内容をGUIで可視化
 - 計測ログ
 - 最新100件
 - 小窓でスクロール
 - csvエクスポート
 - イベントログ
 - 最新100件
 - 小窓でスクロール
 - csvエクスポート
- ①-4 Sender①・②領域
 - 機能のOn/Offの状態
 - 送信レート(映像・音声)
 - マルチキャストグループアドレス(映像・音声)
- ①-6 Receiver①・②領域
 - 機能のOn/Offの状態
 - 受信レート(映像・音声)
 - 設定されているソースIPアドレス・マルチキャストグループアドレス(映像・音声)
 - 映像のWebRTCによる画面
 - 音声のバーメーター
- レイアウト調整機能
 - ①-1(機能領域)・①-2(システム領域)は常時小さく表示する(検証作業時に頻繁に参照しない領域のため)
 - ①-3(PTP領域)と①-4～①-7(Sender/Receiver領域)の境界をドラッグすることで、両領域の表示比率を上下に拡大縮小できる
 - 画面全体を100%表示できるようにする(ブラウザの拡大縮小に依存せず画面内に収まる)

3.4.4. 機能一覧 設定変更・WebGUI②

- PTP設定タブ
 - 本章の「PTP Follower機能」で定めた内容を設定する
 - ログのクリアが可能
 - イベントログの閾値設定
 - $\log_2(1/\text{Hz})$ が整数にならないHz値は入力エラーとする
 - 例: 100Hz、200Hz等は不可(1024Hz、128Hz、64Hz、8Hz、1Hz等の2のべき乗のみ許可)
 - 保存ボタン押下時にバリデーションを行い、不正な値の場合は保存を中断し、該当入力欄を赤枠にしてエラーメッセージ「Hzは2のべき乗の値のみ入力可能です」を表示する
 - エラー時はconfig.jsonへの書き込みは行わない
- Nodeタブ
 - 共通
 - NMOSでのNode名
 - RDSのIPアドレスとポート番号
 - Registration APIのバージョン
 - v1.0
 - v1.1
 - v1.2
 - v1.3
 - Sender
 - RTPのペイロードID
 - マルチキャストグループアドレス
 - ポート番号

- フォーマット
 - 映像
 - 解像度
 - 1920×1080
 - 640×480
 - フレーム/フィールド周波数
 - 59.94
 - 50
 - 走査方式
 - インターレース
 - プログレッシブ
 - カラリメトリ・量子化・クロマフォーマットは下記で固定
 - SDR, 10bit, 4:2:2
 - 音声
 - パケットインターバル
 - 1ms
 - 125us
 - チャンネル数
 - 2ch
 - 4ch
 - 8ch
 - 16ch
 - サンプリングレート・量子化は下記で固定
 - 48KHz, 24bit
- ソースIPアドレスは、メディア面NICの値が「表示」される
- SDPファイルのエクスポート機能
- Receiver
 - 共通
 - NMOSで上位から制御可能だが、本機能で上書き&後取りで設定変更が可能。検証用なので割り切りでOK
 - SDPファイルのインポート機能
 - 映像と音声でそれぞれで設定項目
 - ソースIPアドレス
 - マルチキャストグループアドレス
 - ポート番号
 - RTPペイロードID
 - 映像
 - 解像度
 - 1920×1080
 - 640×480
 - フレーム/フィールド周波数
 - 59.94
 - 50
 - 走査方式
 - インターレース
 - プログレッシブ
 - カラリメトリ・量子化・クロマフォーマットは下記で固定
 - SDR, 10bit, 4:2:2
 - 音声
 - パケットインターバル
 - 1ms
 - 125us
 - チャンネル数
 - 2ch

- 4ch
 - 8ch
 - 16ch
- サンプルレート・量子化は下記で固定
 - 48KHz, 24bit
- システム設定変更タブ
 - マルチキャスト用、制御用、それぞれで使用するNICのIDを紐づける
 - 設定変更後は本システム再起動
 - 3つのNICのIPアドレス関係を変更する
 - 設定変更後はサーバーOS再起動
 - NICの内容は素人ユーザーでも操作できるように、“ifconfig”で表示される、NICのIDを本システムが自動取得して、WebGUIでプルダウンで選択できるようにする
 - サーバーのシャットダウンを行う
 - 設定内容のエクスポート(jsonファイルで設定状態を丸ごと出力)
 - 本項「システム設定」の設定項目を除く全ての設定
 - 設定内容のインポート(上記で出力したjsonファイルで状態復元)
 - 本項「システム設定」の設定項目を除く全ての設定
 - システム設定タブの保存時は、変更内容に応じて下記ポップアップを表示する
 - NIC設定変更 → 「本システムを再起動します」
 - NIC IPアドレス変更 → 「サーバーOSを再起動します」
 - シャットダウン → 「サーバーをシャットダウンします」

4. ④非機能要件

- 4.1. パフォーマンス要件
 - 4.1.1. 定義しない。使用者の自己責任とする
- 4.2. 可用性・信頼性(簡易検証用としての割り切り)
 - 4.2.1. 定義しない。使用者の自己責任とする
- 4.3. セキュリティ要件(社内ローカル前提)
 - 4.3.1. 定義しない。使用者の自己責任とする

5. ⑤アーキテクチャ・技術スタック

5.1. 全体アーキテクチャ

- 5.1.1. Ubuntu Serverの機能範囲内を前提とする
- 5.1.2. NIC①Amber・NIC②Blue・NIC③制御の3NIC構成とする。ただし、NIC②Blueはオプション扱いでなくても基本的な動作に問題がないことを前提とする
- 5.1.3. Docker Composeで3コンテナを管理(第1期は2個)
 - コンテナ① Frontend(Python/FastAPI)
 - コンテナ② PTP Follower
 - コンテナ③ Node(Sender/Receiver)
 - コンテナ③は第2期のSender/Receiverはコンテナ追加で拡張
- 5.1.4. OS起動時はDocker Composeの自動起動で対応

5.2. フロントエンド(Python Webサーバー)

- 5.2.1. PythonでのWebサーバーとする
 - フレームワーク: FastAPI
 - 画面描画: Jinja2テンプレート

- リアルタイム更新:SSE (Server-Sent Events)
- 言語:Python 3.12

5.3. バックエンド(コンテナ構成)

5.3.1. コンテナ② PTP Follower

- ベース:Ubuntu 24.04
- ソフト:linuxptp (ptp4l)
- ネットワーク:hostモード(PTPの制約)
 - これはDockerの通常設定と異なるため、Dockerfileで明示的に設定が必要です。
- 時計の調整について、PTPで取得した時計をMasterとしてOSの時計をSlaveとする。

5.3.2. データストアはDBのディレクトリを作り直接読み書きする

- SQLiteでPTPログを管理
- 設定値はJSONファイルで管理
- リアルタイム状態はメモリ保持

5.4. ミドルウェア選定

- 5.4.1. コンテナ間通信:Docker Compose内部ネットワーク
- 5.4.2. REST API(追加ミドルウェアなし)
- 5.4.3. PTPログ収集 :logs/ptp4l.logファイルをsubprocessでtail読み取り
- 5.4.4. HTTPサーバー :Uvicorn(シングルワーカー)
- 5.4.5. DBアクセス :SQLAlchemy(FastAPI経由)

6. ⑥インターフェース要件

6.1. WebGUI画面一覧

- 6.1.1. 機能
 - 3章に記載
- 6.1.2. トップ画面
 - WebGUI① ダッシュボード
 - トップ画面とする
 - <http://IPアドレス> で表示される画面
- 6.1.3. 遷移設計・画面イメージ
 - 補足資料「検証用MoIP-Node開発_要件定義書_補足資料」、ページ1~3に記載

6.2. フロントエンド／バックエンド間API

① PTP状態系(SSE・GET)

メソッド	エンドポイント	概要
GET/SSE	/api/ptp/status/stream	PTP状態をSSEでリアルタイム配信(1秒更新)
GET	/api/ptp/status	PTP状態を1回取得(ロック状態・GM ID・Source ID・Offset)

② PTP設定系(GET・POST)

メソッド	エンドポイント	概要
GET	/api/ptp/config	現在のPTP設定値を取得
POST	/api/ptp/config	PTP設定値を変更・保存(ドメイン番号・Priority1/2・各Interval)

③ PTPログ系(GET・DELETE)

メソッド	エンドポイント	概要
GET	/api/ptp/log	PTPログ一覧取得(CSVエクスポート用)
DELETE	/api/ptp/log	PTPログ消去

④ システム設定系(GET・POST)

メソッド	エンドポイント	概要
GET	/api/system/config	システム設定値を取得(NIC設定①②③・IPアドレス)
POST	/api/system/config	システム設定値を変更・保存
POST	/api/system/restart	本システム再起動
POST	/api/system/reboot	サーバーOS再起動
POST	/api/system/shutdown	サーバーシャットダウン

⑤ 設定エクスポート／インポート系(GET・POST)

メソッド	エンドポイント	概要
GET	/api/config/export	全設定をJSONファイルでエクスポート
POST	/api/config/import	JSONファイルから全設定をインポート

6.3. 外部システムとのインターフェース

6.3.1. WebGUI

- ポート番号80でのWebGUI

6.3.2. 外部データ連携

- REST API

外部監視系(GET) ※Zabbix等向け

メソッド	エンドポイント	概要
GET	<code>/api/external/ptp/status</code>	PTP状態をJSON形式で出力(外部システム向け)

6.3.3. NMOS

- RDSをstaticでIPアドレス・ポート番号を指定して、IS-04のRegistration APIで自身のNodeのSender、Receiver情報を登録する。
- IS-05に則り、NMOSコントローラーからのReceiverに対する制御を受け取る。
- アウトオブバンドなので、NIC③のみの対応とする

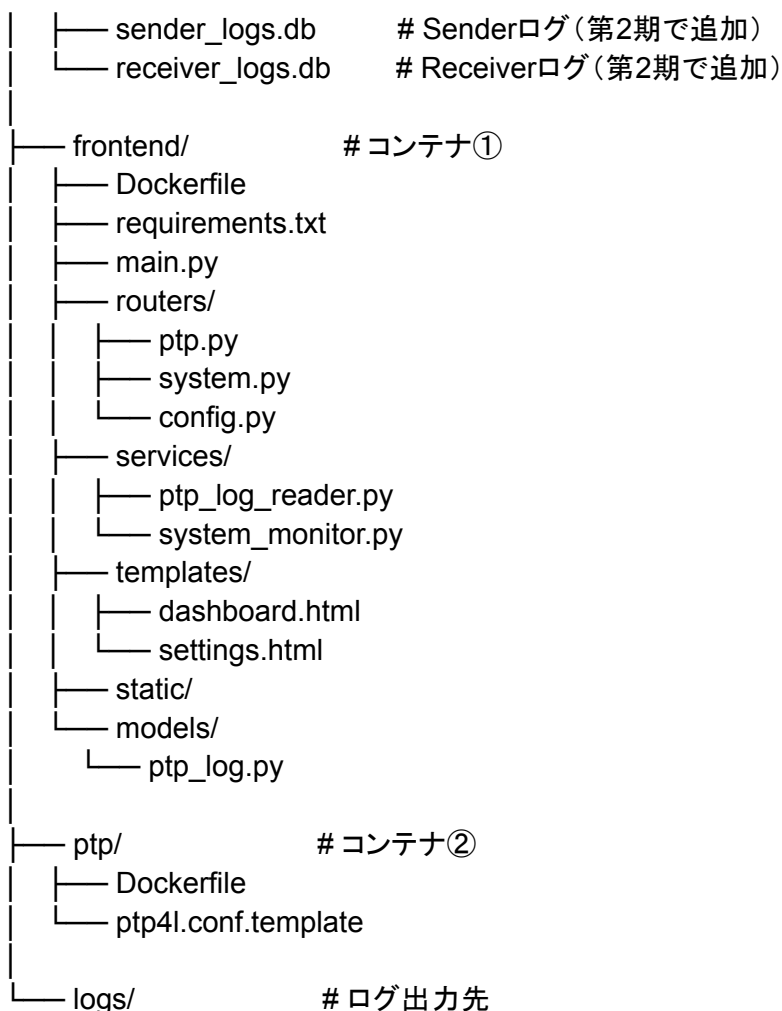
NMOS関係 ★第2期予定

	メソッド	エンドポイント	概要
Sender	GETなど		IS-05に準拠してSDPファイルを別システムに出力 Verは1.0～1.1
Reciver	PATCHなど		IS-05に準拠して別システムからSDPを受信して、メディアNICからIGMP Joinを出力する Verは1.0～1.1
Node	POST		IS-04のRegistration APIでRDSに自身を登録 Verは1.0～1.3まで用意

7. ⑦データ仕様

7.1. ディレクトリ構成

```
moip-node/
├── docker-compose.yml    # Dockerのコンフィグ
├── .env
├── config/               # システム全体のコンフィグ
│   ├── config.json      # ①システム設定(PTP・NIC・Node共通)
│   ├── sdp_config.json  # ②SDP内容管理(第2期・Sender/Receiver)
│   └── ptp4l.conf       # ptp4l用設定(config.jsonから生成)
├── data/                 # DB保存領域
└── ptp_logs.db           # PTP計測・イベントログ(第1期)
```



7.2. コンテナ間通信詳細

7.2.1. コンテナ①→② PTPログ取得(ファイル読み取り方式)

- コンテナ②(pty4l)の標準出力をlogs/pty4l.logにリダイレクトし、コンテナ①(FastAPI)がそのファイルをsubprocessで末尾からtail読み取りする。
 - 共有ボリューム: ./logs を両コンテナにマウント
 - コンテナ②: pty4lの出力を /var/log/pty/pty4l.log に書き出す
 - コンテナ①: /app/logs/pty4l.log を末尾から逐次読み取り、1行ずつパースしてメモリ上の状態変数を更新する

7.2.2. コンテナ①→DB SQLite読み書き(ファイル直接アクセス)

- コンテナ①(FastAPI)がdata/pty_logs.dbにSQLAlchemy経由で直接読み書きする。コンテナは存在せず、data/ディレクトリをコンテナ①にボリュームマウントするだけ。
 - 共有ボリューム: ./data をコンテナ①にマウント
 - アクセス方式: SQLAlchemy(sqlite:///app/data/pty_logs.db)

7.2.3. コンテナ①→② PTP設定変更(ファイル書き換え+コンテナ再起動)

- WebGUIでPTP設定を変更した際、コンテナ①がconfig/pty4l.confを書き換えた後、コンテナ②を再起動して設定を反映させる。
 - 共有ボリューム: ./config を両コンテナにマウント
 - 手順
 - コンテナ①がconfig/pty4l.confを書き換える
 - コンテナ①がDockerソケット経由でコンテナ②を再起動する
 - (/var/run/docker.sock をコンテナ①にマウント)
 - 再起動中はWebGUIのPTP状態表示を「再起動中」と表示する

7.3. DBスキーマ

7.3.1. 使用DB:SQLite(data/ptp_logs.db)

- 1つのDBファイルに全期間のデータを蓄積する。肥大化の懸念はなく(1年運用で約100MB程度)、WebGUIのログクリア機能で定期的に掃除できる。第2期のSender/Receiverのログは別DBファイル(sender_logs.db・receiver_logs.db)として追加する。
- テーブル① ptp_measurement_logs(計測ログ)

カラム名	型	説明
id	INTEGER	主キー・自動採番
recorded_at	TEXT	記録時刻(OS時計・ISO8601形式)
lock_status	INTEGER	PTPロック状態(1=ロック中・0=非ロック)
gm_id	TEXT	GM ID
source_id	TEXT	Source ID
offset_avg_ns	INTEGER	Offset from Master 1分間平均値(ns)
offset_max_ns	INTEGER	Offset from Master 1分間最大値(ns)
offset_min_ns	INTEGER	Offset from Master 1分間最小値(ns)

- テーブル② ptp_event_logs(イベントログ)

カラム名	型	説明
id	INTEGER	主キー・自動採番
recorded_at	TEXT	記録時刻(OS時計・ISO8601形式)
event_type	TEXT	イベント種別(下記参照)
detail	TEXT	詳細情報(JSON文字列)

- event_typeの定義
 - PTP_LOCK_LOST PTPロック外れ
 - OFFSET_THRESHOLD Offset閾値越え
 - PACKET_FREQ_ERROR PTPパケット周波数乱れ
 - GM_ID_CHANGED GM ID変更

7.4. SDPファイルの構造

- ### 7.4.1. 映像用SDPファイル。これになるように.jsonを作る。★はWebGUIで設定する項目。■はシステム設定から動的に変わる項目。これ以外はJSONファイルで静的に設定する

v=0

o=★SONY 3983864702 3983864702 IN IP4 ■10.1.1.2

```

s=★HDCE-TX30 ★30038 ★Cam3 ★Video1
t=0 0
a=group:DUP PRIMARY SECONDARY
m=video ★50020 RTP/AVP ★96
c=IN IP4 ★232.20.101.1/32
a=ts-refclk:ptp=IEEE1588-2008:■BC-8D-1F-FF-FE-06-99-CB:1
a=mediaclock:direct=0
a=source-filter: incl IN IP4 ★232.20.101.1 ■10.1.1.2
a=rtpmap:★96 raw/90000
a=fmtp:★96 width=★1920; height=★1080; exactframerate=★30000/1001;
interlace; sampling=YCbCr-4:2:2; depth=10; colorimetry=BT709; TCS=SDR;
PM=2110GPM; SSN=ST2110-20:2017; TP=2110TPN;
a=mid:PRIMARY
m=video ★50020 RTP/AVP ★96
c=IN IP4 ★232.22.101.1/32
a=ts-refclk:ptp=IEEE1588-2008:■BC-8D-1F-FF-FE-06-99-CB:1
a=mediaclock:direct=0
a=source-filter: incl IN IP4 ★232.22.101.1 ■10.22.103.2
a=rtpmap:★96 raw/90000
a=fmtp:★96 width=★1920; height=★1080; exactframerate=★30000/1001;
interlace; sampling=YCbCr-4:2:2; depth=10; colorimetry=BT709; TCS=SDR;
PM=2110GPM; SSN=ST2110-20:2017; TP=2110TPN;
a=mid:SECONDARY

```

- 7.4.2. 音声用SDPファイル。これになるように.jsonを作る。★はWebGUIで設定する項目。■はシステム設定から動的に変わる項目。これ以外はJSONファイルで静的に設定する

```

v=0
o=★SONY 3983864703 3983864702 IN IP4 ■10.1.1.2
s=★HDCE-TX30 ★30038 ★Cam3 ★Audio Out
t=0 0
a=group:DUP PRIMARY SECONDARY
m=audio ★50030 RTP/AVP ★97
c=IN IP4 ★232.30.101.1/32
a=ts-refclk:ptp=IEEE1588-2008:■BC-8D-1F-FF-FE-06-99-CB:1
a=mediaclock:direct=0
a=source-filter: incl IN IP4 ★232.30.101.1 ■10.1.1.2
a=ptime:★1.0
a=rtpmap:★97 ★L24/48000/8
a=fmtp:★97 channel-order=SMPTE2110.(U08);
a=mid:PRIMARY
m=audio ★50030 RTP/AVP ★97
c=IN IP4 ★232.33.101.1/32
a=ts-refclk:ptp=IEEE1588-2008:■BC-8D-1F-FF-FE-06-99-CB:1
a=mediaclock:direct=0
a=source-filter: incl IN IP4 ★232.33.101.1 ■10.22.103.2
a=ptime:★1.0
a=rtpmap:★97 ★L24/48000/8
a=fmtp:★97 channel-order=SMPTE2110.(U08);
a=mid:SECONDARY

```

7.5. 設定JSONの構造

7.5.1. config/config.jsonの構造

```
{
  "ptp": {
    "domain": 127,
    "priority1": 128,
    "priority2": 128,
    "sync_interval": -7,
    "announce_interval": 1,
    "delay_request_interval": 0,
    "lock_threshold_ns": 500,
    "lock_stable_sec": 3,
    "offset_alert_threshold_ns": 500
  },
  "system": {
    "media_nic_amber": "eth0",
    "media_nic_blue": "eth1",
    "control_nic": "eth2"
  },
  "node": {
    "node_name": "moip-node-01",
    "rds_ip": "192.168.1.100",
    "rds_port": 3211,
    "registration_api_version": "v1.3"
  }
}
```

7.5.2. sdp_config.jsonフォーマット(SDP内容管理)

- 前述のSDPファイルのパラメータを、静的・動的関係なく全て包括する

```
{
  "sender": [
    {
      "id": 1,
      "video": {
        "_comment": "=== o行 (origin) ★=WebGUI設定 ■=動的 それ以外=静的固定 ===",
        "origin_username": "SONY",
        "origin_session_id": "3983864702",
        "origin_session_version": "3983864702",
        "_comment2": "=== s行 (session name) ===",
        "session_device_name": "HDCE-TX30",
        "session_node_id": "30038",
        "session_source_name": "Cam3",
        "session_stream_name": "Video1",
        "_comment3": "=== m行 (media) ・c行 (multicast) ===",
        "port": 50020,
        "payload_id": 96,
        "multicast_address_amber": "232.20.101.1",
        "multicast_address_blue": "232.22.101.1",
        "_comment4": "=== a=fmtp 映像パラメータ ===",
        "width": 1920,

```

```

    "height": 1080,
    "exactframerate": "30000/1001",
    "scan": "interlace",
    "_comment5": "=== 静的固定値(変更不要) ===",
    "sampling": "YCbCr-4:2:2",
    "depth": 10,
    "colorimetry": "BT709",
    "tcs": "SDR",
    "pm": "2110GPM",
    "ssn": "ST2110-20:2017",
    "tp": "2110TPN"
  },
  "audio": {
    "_comment": "=== o行 (origin) ===",
    "origin_username": "SONY",
    "origin_session_id": "3983864703",
    "origin_session_version": "3983864702",
    "_comment2": "=== s行 (session name) ===",
    "session_device_name": "HDCE-TX30",
    "session_node_id": "30038",
    "session_source_name": "Cam3",
    "session_stream_name": "Audio Out",
    "_comment3": "=== m行 (media)・c行 (multicast) ===",
    "port": 50030,
    "payload_id": 97,
    "multicast_address_amber": "232.30.101.1",
    "multicast_address_blue": "232.33.101.1",
    "_comment4": "=== a=ptime・a=rtpmap 音声パラメータ ===",
    "ptime": 1.0,
    "channels": 8,
    "_comment5": "=== 静的固定値(変更不要) ===",
    "encoding": "L24",
    "sampling_rate": 48000,
    "channel_order": "SMPTE2110.(U08)"
  }
},
{
  "id": 2,
  "video": {
    "origin_username": "SONY",
    "origin_session_id": "3983864704",
    "origin_session_version": "3983864704",
    "session_device_name": "HDCE-TX30",
    "session_node_id": "30039",
    "session_source_name": "Cam4",
    "session_stream_name": "Video2",
    "port": 50024,
    "payload_id": 96,
    "multicast_address_amber": "232.20.102.1",
    "multicast_address_blue": "232.22.102.1",
    "width": 1920,
    "height": 1080,
    "exactframerate": "30000/1001",
    "scan": "interlace",

```



```
"sampling": "YCbCr-4:2:2",
"depth": 10,
"colorimetry": "BT709",
"tcs": "SDR",
"pm": "2110GPM",
"ssn": "ST2110-20:2017",
"tp": "2110TPN"
},
"audio": {
  "origin_username": "SONY",
  "origin_session_id": "3983864705",
  "origin_session_version": "3983864704",
  "session_device_name": "HDCE-TX30",
  "session_node_id": "30039",
  "session_source_name": "Cam4",
  "session_stream_name": "Audio Out2",
  "port": 50034,
  "payload_id": 97,
  "multicast_address_amber": "232.30.102.1",
  "multicast_address_blue": "232.33.102.1",
  "ptime": 1.0,
  "channels": 8,
  "encoding": "L24",
  "sampling_rate": 48000,
  "channel_order": "SMPTE2110.(U08)"
}
},
],
"receiver": [
  {
    "id": 1,
    "video": {
      "port": 50020,
      "payload_id": 96,
      "multicast_address_amber": "232.20.101.1",
      "multicast_address_blue": "232.22.101.1",
      "width": 1920,
      "height": 1080,
      "exactframerate": "30000/1001",
      "scan": "interlace",
      "sampling": "YCbCr-4:2:2",
      "depth": 10,
      "colorimetry": "BT709",
      "tcs": "SDR"
    },
    "audio": {
      "port": 50030,
      "payload_id": 97,
      "multicast_address_amber": "232.30.101.1",
      "multicast_address_blue": "232.33.101.1",
      "ptime": 1.0,
      "channels": 8,
      "encoding": "L24",
      "sampling_rate": 48000,
```

```

        "channel_order": "SMPTE2110.(U08)"
    }
},
{
    "id": 2,
    "video": {
        "port": 50024,
        "payload_id": 96,
        "multicast_address_amber": "232.20.102.1",
        "multicast_address_blue": "232.22.102.1",
        "width": 1920,
        "height": 1080,
        "exactframerate": "30000/1001",
        "scan": "interlace",
        "sampling": "YCbCr-4:2:2",
        "depth": 10,
        "colorimetry": "BT709",
        "tcs": "SDR"
    },
    "audio": {
        "port": 50034,
        "payload_id": 97,
        "multicast_address_amber": "232.30.102.1",
        "multicast_address_blue": "232.33.102.1",
        "ptime": 1.0,
        "channels": 8,
        "encoding": "L24",
        "sampling_rate": 48000,
        "channel_order": "SMPTE2110.(U08)"
    }
}
]
}

```

7.6. ptp4lログフォーマット

7.6.1. ptp4lの出力ログ例

```

ptp4l[12345.678]: port 1: UNCALIBRATED to SLAVE on MASTER_CLOCK_SELECTED
ptp4l[12345.679]: selected best master clock 001b21.ffe.000001
ptp4l[12346.680]: master offset -23 s2 freq -1234 path delay 456
ptp4l[12347.681]: master offset 45 s2 freq -1230 path delay 458
ptp4l[12348.682]: master offset -12 s2 freq -1232 path delay 455

```

7.6.2. 各フィールドの意味

ptp4l[タイムスタンプ]: master offset [値] s[状態] freq [値] path delay [値]

- master offset: Offset from Master(ns単位) ← 最重要

- s0 / s1 / s2: 同期状態

- s0 = UNLOCKED(非ロック)

- s1 = CALIBRATING(調整中)

- s2 = LOCKED(ロック済み)

- freq: 周波数補正值(参考値)

- path delay: パスディレイ(ns単位・参考値)

7.6.3. コンテナ①でのパース対象ログパターン

パターンA Offset取得

master offset\s+(-?\d+)\s+s(\d+)\s+freq\s+(-?\d+)\s+path delay\s+(\d+)

→ offset_ns(整数・ns単位)、sync_state(0/1/2)を取得

パターンB GM ID取得

selected best master clock\s+([0-9a-f]{6})\.\xffe\.[0-9a-f]{6}

→ gm_id を取得

パターンC Source ID取得(Announce送信元MACアドレスを使用)

port \d+: (\w+) to (\w+) on

→ port_state(状態遷移)を取得し、

Announce送信元MACアドレスをsource_idとして使用

7.6.4. メモリ上で保持するPTP状態変数

```
{
  "lock_status": false,
  "gm_id": "unknown",
  "source_id": "unknown",
  "offset_current_ns": 0,
  "offset_avg_ns": 0,
  "offset_max_ns": 0,
  "offset_min_ns": 0,
  "offset_samples": [],
  "lock_stable_count": 0,
  "last_updated": "2026-07-02T00:00:00"
}
```

フィールド	説明
lock_status	PTPロック状態(true=ロック中)
gm_id	GM ID(BMCAで決定されたGrandmaster ID)
source_id	Source ID(Announce送信元MACアドレス)
offset_current_ns	最新のOffset from Master(ns)
offset_avg_ns	1秒間の平均値(WebGUI・外部API配信用)
offset_max_ns	1秒間の最大値
offset_min_ns	1秒間の最小値
offset_samples	1秒分のサンプル蓄積バッファ(平均・最大・最小算出用)
lock_stable_count	PTPロック判定カウンター(s2状態が連続3秒でlock_status=true)
last_updated	最終更新時刻(ISO8601形式)

7.7. SSEデータフォーマット

7.7.1. SSEは/api/ptp/status/streamエンドポイントから1秒ごとに以下の形式で配信する

7.7.2. SSEストリームの形式

```
data: {"lock_status": true, "gm_id": "001b21.ffe.000001", "source_id":  
"00:1b:21:ff:fe:00:00:02", "offset_avg_ns": -23, "offset_max_ns": 45, "offset_min_ns": -67,  
"offset_current_ns": -12, "last_updated": "2026-07-02T12:34:56"}
```

```
data: {"lock_status": true, "gm_id": "001b21.ffe.000001", "source_id":  
"00:1b:21:ff:fe:00:00:02", "offset_avg_ns": -20, "offset_max_ns": 40, "offset_min_ns": -60,  
"offset_current_ns": -18, "last_updated": "2026-07-02T12:34:57"}
```

各フィールドの説明

フィールド	型	説明
lock_status	boolean	PTPロック状態 (true=ロック中・false=非ロック)
gm_id	string	GM ID (BMCAで決定されたGrandmaster ID)
source_id	string	Source ID (Announce送信元MACアドレス)
offset_avg_ns	integer	直近1秒間のOffset平均値 (ns)
offset_max_ns	integer	直近1秒間のOffset最大値 (ns)
offset_min_ns	integer	直近1秒間のOffset最小値 (ns)
offset_current_ns	integer	最新のOffset値 (ns)
last_updated	string	最終更新時刻 (ISO8601形式)

7.7.3. PTP未接続・初期状態の場合

```
{  
  "lock_status": false,  
  "gm_id": "unknown",  
  "source_id": "unknown",  
  "offset_avg_ns": 0,  
  "offset_max_ns": 0,  
  "offset_min_ns": 0,  
  "offset_current_ns": 0,  
  "last_updated": "2026-07-02T00:00:00"  
}
```

7.7.4. WebGUI側での受信処理イメージ

```
// dashboard.html でのSSE受信  
const evtSource = new EventSource("/api/ptp/status/stream");  
evtSource.onmessage = (event) => {  
  const data = JSON.parse(event.data);  
  document.getElementById("lock_status").textContent  
    = data.lock_status ? "● LOCKED" : "● UNLOCKED";  
  document.getElementById("offset_avg").textContent  
    = data.offset_avg_ns + " ns";  
};
```

7.8. 外部APIレスポンス形式

7.8.1. 3章の外部API定義（Zabbix向け）と7-6のSSEフォーマットを踏まえた、
/api/external/ptp/statusのレスポンス仕様です。

7.8.2. 外部APIレスポンス形式

- 外部監視システム（Zabbix等）向けの /api/external/ptp/status エンドポイントのレスポンス形式を定義する。
- リクエスト
GET /api/external/ptp/status
- レスポンス（JSON形式）

```
{
  "lock_status": true,
  "gm_id": "001b21.ffe.000001",
  "source_id": "00:1b:21:ff:fe:00:00:02",
  "offset_avg_ns": -23,
  "offset_max_ns": 45,
  "offset_min_ns": -67,
  "offset_current_ns": -12,
  "last_updated": "2026-07-02T12:34:56"
}
```
- 各フィールドの説明

フィールド	型	説明
lock_status	boolean	PTPロック状態（true=ロック中・false=非ロック）
gm_id	string	GM ID（BMCAで決定されたGrandmaster ID）
source_id	string	Source ID（Announce送信元MACアドレス）
offset_avg_ns	integer	直近1秒間のOffset平均値（ns）
offset_max_ns	integer	直近1秒間のOffset最大値（ns）
offset_min_ns	integer	直近1秒間のOffset最小値（ns）
offset_current_ns	integer	最新のOffset値（ns）
last_updated	string	最終更新時刻（ISO8601形式）

7.8.3. PTP未接続・初期状態の場合

```
{
  "lock_status": false,
  "gm_id": "unknown",
  "source_id": "unknown",
  "offset_avg_ns": 0,
  "offset_max_ns": 0,
  "offset_min_ns": 0,
  "offset_current_ns": 0,
  "last_updated": "2026-07-02T00:00:00"
}
```

7.8.4. 仕様上の補足

- リクエストを受けた時点のメモリ上の最新値をそのまま返す
- 認証なし(社内ローカル前提・検証用のため)
- HTTPステータスコードは正常時200を返す
- Content-Type: application/json

7.8.5. 第2期拡張予定

- Sender/Receiverの状態についても、以下のエンドポイントを第2期で追加する想定とする。
 - GET /api/external/sender/status
 - GET /api/external/receiver/status
- 具体的なレスポンス形式は第2期要件定義時に別途定義する。

8. ⑧構築手順書

8.1. 開発完了後の構築手順は別紙とする

8.1.1. TBD